



Predictions + Train and Test set

⚙ Status	In progress
🕒 Last edited time	March 20, 2026 5:15 PM
🔔 Priority	High

Good question — this is actually one of the **most important methodological decisions** in your thesis.

You are at the point where **rule discovery becomes rule-based prediction**, and that needs to be defined carefully.

Your intuition about using the **median (or mean) of the target values covered by the rule** is **very reasonable**, and in fact it is one of the standard approaches in rule-based regression.

Let me explain the **best approach for your case**.

1 What CarenR Actually Gives You

From what you described, CarenR produces rules like:

```
IF conditions THEN subgroup with target distribution
```

Example:

```
IF SpringTemp > 18 AND Rainfall < 40  
THEN wine production distribution:  
mean = 62  
median = 60  
sd = 10
```

So a rule does **not directly output a prediction**, it outputs a **distribution of the target variable**.

Therefore, for prediction you must define a **mapping from rule** → **predicted value**.

2 Best Prediction Strategy

For each rule compute a **representative value** from the target distribution.

The best options are:

Method	Prediction value	Comment
Median	median(target in rule)	Most robust
Mean	mean(target in rule)	Sensitive to outliers
Conditional mean	mean weighted by support	Sometimes used
Distribution sampling	draw from distribution	Rarely used in theses

My recommendation

Use **median**.

Why?

- robust to outliers
 - stable with small sample sizes
 - easy to interpret
 - very common in rule-based regression.
-

3 Prediction Pipeline

Step 1 — Split dataset

Since your dataset is **1933–2022**, you should avoid random splits because of temporal dependence.

Use **time-based split**.

Example:

Set	Years
Train	1933–2000

Set	Years
Test	2001–2022

This is much stronger scientifically.

Step 2 — Discover rules on training set

Run:

```
CarenR(train_data)
```

This produces rules.

Step 3 — Compute rule prediction value

For each rule calculate:

```
median(target values in training set covered by rule)
```

Example:

Rule	Median production
R1	58
R2	72
R3	49

Step 4 — Predict test observations

For each observation in test set:

1. Find rule(s) whose conditions are satisfied
2. Assign the rule prediction value.

Example:

```
IF rule 2 applies → predicted production = 72
```

4 What if Multiple Rules Apply?

Use **one of these strategies**.

Option A (simplest)

First matching rule.

This mimics **RIPPER behaviour**.

Option B (better for CarenR)

Average predictions of all matching rules.

Example:

```
Rule1 → 58  
Rule2 → 72  
Prediction = 65
```

Option C (very strong academically)

Weight predictions by **rule support**.

```
prediction =  
(sum rule_prediction * rule_support)  
/  
(sum rule_support)
```

5 What if No Rule Covers the Observation?

You need a **default prediction**.

Use:

```
median(target_train)
```

This is standard practice.

6 Evaluation Metrics

Once predictions are computed on the **test set**, calculate:

Metric	Why
RMSE	penalizes large errors
MAE	robust error measure
R ²	variance explained

Example:

Model	RMSE	MAE	R ²
CarenR rules	0.82	0.64	0.71
Regression Tree	0.79	0.61	0.73
Random Forest	0.68	0.52	0.81

7 Why This Is a Good Thesis Method

This setup:

- keeps **rule discovery independent**
- converts rules into **predictive model**
- allows **direct comparison with ML models**

Most rule-discovery theses use exactly this approach.

8 Very Important: Rule Coverage

Also report:

Metric	Meaning
coverage	% of observations covered by rules
avg rules per observation	rule overlap
default predictions	cases with no rule

These give insight into rule quality.

9 How RIPPER Fits

RIPPER already produces predictions.

So evaluation is easier:

```
predict(JRip_model, test_data)
```

But remember:

RIPPER predictions are **class predictions**, not continuous values.

So you may map classes to:

```
class median production
```

Example:

Class	Median production
Low	48
Medium	61
High	74

10 Final Experimental Pipeline

Your thesis experiment becomes:

Model	Target	Prediction method
CarenR	continuous	median of rule
RIPPER	discretized	class median
Regression Tree	continuous	tree prediction
Random Forest	continuous	RF prediction

My strongest recommendation

Use:

- **median of rule distribution**
- **time-based train/test split**
- **weighted rule averaging if multiple rules apply**

This will make your methodology **very defensible academically**.

If you want, I can also show you the **exact R implementation for CarenR rule prediction**, because the prediction step usually needs a small custom function.

Yes — **in practice all the algorithms in your setup can produce predictions**, and therefore **RMSE and MAE can be computed**. But there is one small methodological detail you should handle carefully.

Let's clarify it precisely.

1. CarenR

CarenR produces rules associated with a **distribution of the continuous target**.

Example rule:

```
IF Temp > 18 AND Rainfall < 40  
THEN production distribution  
mean = 62  
median = 60  
sd = 10
```

To compute prediction error metrics, you must convert that distribution into a **single predicted value**.

Recommended:

```
prediction = median(target values covered by rule in training set)
```

Then:

```
RMSE = sqrt(mean((y_test - y_pred)^2))  
MAE = mean(|y_test - y_pred|)
```

So yes — CarenR can be evaluated with **RMSE and MAE**.

2. RIPPER

RIPPER is a **classification algorithm**.

Example rule:

```
IF Temp > 18 AND Rainfall < 40 → class = HighProduction
```

So it predicts **classes**, not continuous values.

To compute RMSE/MAE you must map classes back to numbers.

Example:

Class	Median production
Low	48
Medium	61
High	74

Prediction pipeline:

```
RIPPER → class prediction  
class → median production
```

Then compute RMSE/MAE.

This is a standard trick used in many studies comparing **classification vs regression models**.

3. Decision Tree (Regression Tree)

Regression trees naturally output a continuous prediction.

Example leaf:

```
Leaf prediction = mean production of training samples in leaf
```

So evaluation is straightforward:

```
RMSE  
MAE
```


R^2

No conversion required.

4. Random Forest / Boosting (optional)

Same as regression tree:

continuous prediction → compute RMSE and MAE directly.

5. Why RMSE and MAE Are Good Here

Because your **true target is continuous wine production**, these metrics evaluate how well the models approximate the true value.

Recommended metrics:

Metric	Reason
RMSE	penalizes large errors
MAE	robust to outliers
R^2	interpretable variance explained

MAE is particularly good when:

- data is small
- outliers exist

Your dataset likely has both.

6. One Important Additional Metric for Rules

Besides RMSE/MAE, rule-based models should also report **coverage**.

Example:

Model	RMSE	MAE	Coverage
CarenR	0.82	0.63	76%
RIPPER	0.88	0.67	72%

Model	RMSE	MAE	Coverage
Tree	0.79	0.61	100%

Coverage tells you:

| How much of the dataset is explained by explicit rules.

This is important because:

- rule discovery methods often **do not cover all observations**.

7. The Key Takeaway

Yes — you can evaluate **all models with RMSE and MAE**, but the prediction pipelines differ:

Model	Prediction source
CarenR	median of rule target distribution
RIPPER	class → median production
Regression Tree	leaf prediction
Random Forest	ensemble prediction

So the evaluation becomes **comparable across models**.

8. Final thesis-style interpretation

Your evaluation will answer two questions:

1. Predictive performance

| How well do the discovered rules approximate the true production values?

1. Interpretability

| Do rule-based models extract meaningful and compact explanations of wine production patterns?

This combination (prediction + interpretation) is exactly what reviewers expect in a rule discovery thesis.

If you want, I can also show you something **extremely useful for your thesis**:

How to build a **very clean experimental pipeline in R (train/test + all models)** in about **30 lines of code**.

Coverage is simply **how many observations a rule (or rule set) applies to**.

Coverage of a single rule

Formula:

$$\text{Coverage}(\text{rule}) = \frac{\text{\text{\# observations satisfying the rule}}}{\text{\text{\# total \# observations}}}$$

Example:

Dataset size = **120 years**

Rule:

```
Temp > 18 AND Rainfall < 40
```

If **30 years** satisfy the rule:

$$\text{Coverage} = \frac{30}{120} = 0.25$$

So the rule covers **25% of the dataset**.

Coverage of the rule set

This measures how much of the dataset is explained by **at least one rule**.

Coverage(rule\ set) =

$$\frac{\text{\text{\# observations covered by } \geq 1 \text{ rule}}}{\text{\text{\# total observations}}}$$

Example:

Observation	Rule1	Rule2
Year1	✓	
Year2		✓
Year3	✓	✓

Covered observations = **3**

If dataset has **10 observations**:

$$\text{Coverage} = \frac{3}{10} = 30\%$$

Simple R idea

For one rule:

```
coverage <- mean(temp > 18 & rainfall < 40)
```

Because TRUE = 1 and FALSE = 0.

For rule set:

```
covered <- (rule1 | rule2 | rule3)
coverage <- mean(covered)
```

What to report in your thesis

Example table:

Model	# Rules	Coverage
CarenR	6	78%
RIPPER	8	72%
Tree rules	12	100%

Interpretation:

- Higher coverage → rules explain more of the data
- Lower coverage → rules describe **specific subgroups**

Both are acceptable depending on the method.

If you want, I can also show you a **very good metric used in subgroup discovery papers (WRAcc)** that combines **coverage and accuracy** and is often reported together with coverage.

Here are the **coverage formulas written in Notion-friendly format** (using LaTeX blocks you can paste directly in Notion).

1 Coverage of a Single Rule

Notion formula block:

$$Coverage(R) = \frac{|\{x \in D : R(x) = TRUE\}|}{|D|}$$

Meaning:

- D = dataset
- R(x) = rule condition
- numerator = number of observations satisfying the rule
- denominator = total observations

2 Coverage of a Rule Set

```

$$
Coverage(\mathcal{R}) = \frac{|\{x \in D : \exists R_i \in \mathcal{R} \text{ such that } R_i(x) = TRUE\}|}{|D|}
$$

```

Meaning:

- $\mathcal{R}$ = set of rules
- observation counted if **at least one rule applies**

3 Support of a Rule (often used together)

```

$$
Support(R) = \frac{|\{x \in D : R(x)=TRUE \wedge y \in T\}|}{|D|}
$$

```

Where:

- y = target variable
- T = target subgroup or value range

4 Prediction Coverage (useful for your thesis)

If a rule produces a prediction:

```
$$
PredictionCoverage = \frac{|\{x \in D : \exists R_i \text{ producing prediction for } x\}|}{|D|}
$$
```

This measures:

| percentage of observations that receive a rule-based prediction.

5 Simple Version (for explanation text)

```
$$
Coverage = \frac{\text{Number of observations satisfying the rule}}{\text{Total number of observations}}
$$
```

Example you could include in your thesis

```
$$
Coverage(R_1) = \frac{30}{120} = 0.25
$$
```

Meaning the rule explains **25% of the dataset**.

If you want, I can also give you **two more formulas that rule-learning papers almost always include** with coverage:

- **Confidence**
- **WRAcc (Weighted Relative Accuracy)**

These are **very useful for your thesis evaluation section**.